Machine Learning Notes
Preliminary Introduction to Deep Learning

Vitor N. Cabral de Oliveira
vnco@cin.ufpe.br

June 4, 2026

This notebook serves as a comprehensive repository for my Machine Learning studies, with a primary focus on Deep Learning. Key topics covered include:

- Artificial Neural Networks (ANNs) – Fundamentals and architectures with mathematical explaination.

Contents

Artificial Neural Networks

Building on the probabilistic and statistical learning foundations, we now introduce artificial neural networks (ANNs) - a flexible class of models that can approximate complex, nonlinear functions. We begin with the simplest unit, the perceptron, and progress to multilayer networks trained via backpropagation.

The Perceptron

Definition 0.1 (Perceptron) *A **perceptron** is a binary classifier that maps an input vector* $\mathbf{x} = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$ *to an output* $o \in \{0, 1\}$*:*

$$o = \phi\left(w_0 + \sum_{i=1}^{d} w_i x_i\right), \qquad \phi(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$

The parameters w_i *are **weights** and* w_0 *is the **bias**. By augmenting* $\mathbf{x}$ *with a constant* $x_0 = 1$*, we write* $\mathbf{w} = (w_0, \ldots, w_d)^\top$ *and* $o = \mathbf{1}\{\mathbf{w} \cdot \mathbf{x} > 0\}$*.*

The perceptron defines a linear decision boundary $\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} = 0\}$ (see Fig. 1). It can only represent *linearly separable* functions; the XOR problem (Fig. 2) is the classic counterexample where no single hyperplane can partition the activation states.

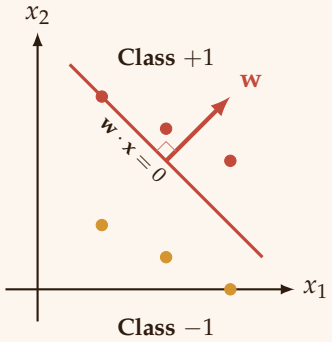

Figure 1: Geometric representation of a linear decision boundary in $\mathbb{R}^2$ where the weight vector $\mathbf{w}$ acts orthogonal to the separating hyperplane.

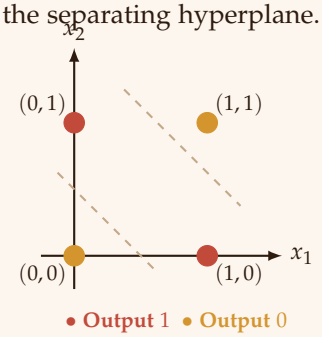

Figure 2: The XOR non-linear separability dilemma. No single linear hyperplane $\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} = 0\}$ can isolate the heterogeneous activation pairs simultaneously.

Learning Rules

Perceptron Training Rule

Given a training set $\mathcal{D} = \{(\mathbf{x}_d, t_d)\}_{d=1}^N$, the perceptron updates weights when a misclassification occurs:

$$w_i \leftarrow w_i + \eta\,(t_d - o_d)\,x_{i,d},$$

with learning rate $\eta > 0$. This converges if the data are linearly separable.

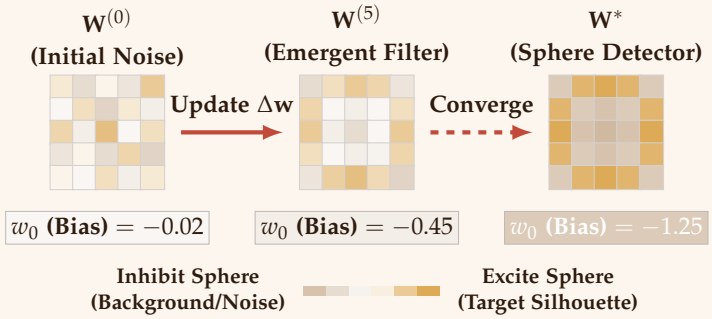

Figure 3: Spatial evolution of perceptron weight updates during shape classification. Over training iterations, the weight matrix transitions from unstructured noise ($\mathbf{W}^{(0)}$) to an interpretable morphology ($\mathbf{W}^*$). The converged state functions as a spatial template matcher, where pixels tracking the circular boundary develop strong excitatory values (gold) while the surrounding corners and center become strongly inhibitory (muted earth tones).

Delta Rule and Gradient Descent

For a linear unit $o = \mathbf{w} \cdot \mathbf{x}$ (no threshold), the squared error is

$$E(\mathbf{w}) = \frac{1}{2} \sum_{d=1}^{N} (t_d - o_d)^2.$$

The gradient is $\nabla E = -\sum_d (t_d - o_d)\mathbf{x}_d$, yielding the batch gradient descent update:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{d=1}^{N} (t_d - o_d)\mathbf{x}_d.$$

Remark 0.1 (Connection to MLE) *For a linear regression model $y = \mathbf{w}^\mathsf{T}\mathbf{x} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the negative log-likelihood is $\frac{1}{2\sigma^2}\sum(y_i - \mathbf{w}^\mathsf{T}\mathbf{x}_i)^2 + \text{const}$. Minimising the squared error is therefore equivalent to maximum likelihood estimation under a Gaussian noise model.*

Stochastic Gradient Descent (SGD)

For large datasets, we use stochastic updates:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i(\mathbf{w}), \qquad i \sim \text{Uniform}\{1, \ldots, N\},$$

or mini-batch updates. SGD introduces noise that can help escape shallow local minima.

Activation Functions

For multilayer networks, we need nonlinear, differentiable activation functions. Common choices are:

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$, outputs in $(0, 1)$. Smooth but saturates for large $|z|$, causing vanishing gradients.
- **Hyperbolic tangent (tanh)**: $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, outputs in $(-1, 1)$. Zero-centered, which helps optimisation, but still saturates.
- **Rectified Linear Unit (ReLU)**: $\text{ReLU}(z) = \max(0, z)$. Non-saturating for $z > 0$, computationally efficient, but can cause "dying neurons".
- **Leaky ReLU**: $\text{LeakyReLU}(z) = \max(\alpha z, z)$ with a small slope α (e.g., 0.01) for $z < 0$, mitigating the dying ReLU problem.

Definition 0.2 (Sigmoid)

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

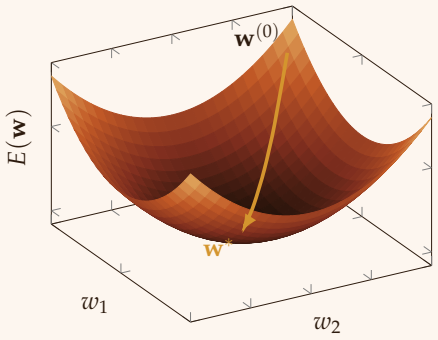

Figure 4: 3D Loss Surface

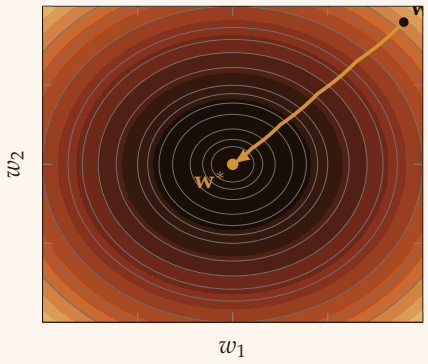

Figure 5: 2D Contour Optimization

Definition 0.3 (Hyperbolic Tangent (Tanh))

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh'(z) = 1 - \tanh^2(z).$$

Definition 0.4 (Rectified Linear Unit (ReLU))

$$ReLU(z) = \max(0, z), \quad ReLU'(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$

ReLU is widely used because it avoids vanishing gradients for positive inputs, though it can cause "dying ReLU" problems.

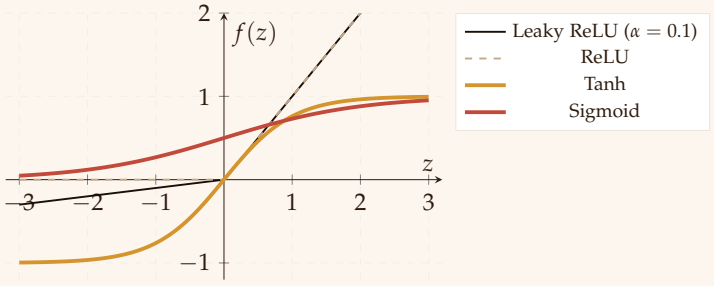

Figure 6: Common activation networks used in neural networks. Sigmoid and tanh saturate, while ReLU and Leaky ReLU are piecewise linear.

Multilayer Perceptron and Backpropagation

A multilayer perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. Let $\mathbf{o}^{(\ell)}$ denote the activations of layer ℓ, with $\mathbf{o}^{(0)} = \mathbf{x}$. For each layer $\ell = 1, \ldots, L$,

$$\mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{o}^{(\ell)} = f^{(\ell)}(\mathbf{a}^{(\ell)}),$$

where $f^{(\ell)}$ is an activation function (applied elementwise), $\mathbf{W}^{(\ell)}$ is the weight matrix, and $\mathbf{b}^{(\ell)}$ the bias vector.

Backpropagation Derivation

We minimise a loss function $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ where $\hat{\mathbf{y}} = \mathbf{o}^{(L)}$. For a single example with target $\mathbf{t}$, define the error at the output layer as the derivative of $\mathcal{L}$ with respect to the pre-activation $\mathbf{a}^{(L)}$:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} = \nabla_{\hat{\mathbf{y}}}\mathcal{L} \odot f^{(L)'}(\mathbf{a}^{(L)}),$$

where $\odot$ is elementwise multiplication. For the squared loss $\mathcal{L} = \frac{1}{2}\|\mathbf{t} - \hat{\mathbf{y}}\|^2$, we have $\nabla_{\hat{\mathbf{y}}}\mathcal{L} = \hat{\mathbf{y}} - \mathbf{t}$.

For earlier layers, the chain rule gives

$$\delta^{(\ell)} = \left((\mathbf{W}^{(\ell+1)})^\top \delta^{(\ell+1)}\right) \odot f^{(\ell)'}(\mathbf{a}^{(\ell)}), \quad \ell = L-1, \ldots, 1.$$

The gradients with respect to weights and biases are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)} (\mathbf{o}^{(\ell-1)})^{\mathsf{T}}, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \boldsymbol{\delta}^{(\ell)}.$$

We then update parameters using SGD (or its variants).

Algorithm 1: Backpropagation (online / mini-batch)

Initialize all weights $\mathbf{W}^{(\ell)}$ and biases $\mathbf{b}^{(\ell)}$ (e.g., Xavier/He).
for each mini-batch $\mathcal{B}$ **do**
  for $\ell = 1$ to L **do**
    Forward: $\mathbf{a}^{(\ell)} = \mathbf{W}^{(\ell)} \mathbf{o}^{(\ell-1)} + \mathbf{b}^{(\ell)}$, $\mathbf{o}^{(\ell)} = f^{(\ell)}(\mathbf{a}^{(\ell)})$
  end for
  Compute loss $\mathcal{L}(\mathbf{o}^{(L)}, \mathbf{t})$ for the batch.
  $\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{o}^{(L)}} \mathcal{L} \odot f^{(L)\prime}(\mathbf{a}^{(L)})$
  for $\ell = L - 1$ down to 1 **do**
    $\boldsymbol{\delta}^{(\ell)} = \left((\mathbf{W}^{(\ell+1)})^{\mathsf{T}} \boldsymbol{\delta}^{(\ell+1)} \right) \odot f^{(\ell)\prime}(\mathbf{a}^{(\ell)})$
  end for
  for $\ell = 1$ to L **do**
    $\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{1}{|\mathcal{B}|} \sum \boldsymbol{\delta}^{(\ell)} (\mathbf{o}^{(\ell-1)})^{\mathsf{T}}$
    $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{1}{|\mathcal{B}|} \sum \boldsymbol{\delta}^{(\ell)}$
  end for
end for

Advanced Optimization Methods

Plain SGD often converges slowly. Modern deep learning uses adaptive methods:

- **Momentum**: accumulates a velocity term $v_t = \gamma v_{t-1} + \eta \nabla \mathcal{L}$, damping oscillations.
- **Adam** (Adaptive Moment Estimation): maintains per-parameter learning rates using first and second moments of gradients.
- **Learning rate schedules**: step decay, cosine annealing, warmup.

Remark 0.2 *Adam is often the default choice for training deep networks, though SGD with momentum can generalise better in some tasks.*

Regularization for Neural Networks

Deep models have many parameters and easily overfit. Common regularization techniques include:

L2 Regularization (Weight Decay)

Adds a penalty $\frac{\lambda}{2}\|\mathbf{w}\|_2^2$ to the loss. From a Bayesian perspective, this corresponds to a Gaussian prior on weights (MAP estimation).

Dropout

Randomly sets a fraction p of neurons to zero during each training forward pass. This prevents co-adaptation and can be interpreted as training an ensemble of subnetworks.

Batch Normalization (BatchNorm)

Normalizes layer inputs to have zero mean and unit variance across the mini-batch. It accelerates training and acts as a regularizer. For a mini-batch $\mathcal{B}$, compute

$$\mu_{\mathcal{B}} = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} a_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(a_i-\mu_{\mathcal{B}})^2, \quad \hat{a}_i = \frac{a_i-\mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2+\epsilon}}, \quad \tilde{a}_i = \gamma\hat{a}_i+\beta.$$

Early Stopping

Monitor validation error and stop training when it stops improving; this implicitly regularizes by limiting the effective capacity.

Bias–Variance Tradeoff in Neural Networks

Recall from statistical learning the decomposition for squared loss:

$$\mathbb{E}_{\mathcal{D}}[R(h_{\mathcal{D}})] = \underbrace{(\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}]-h^*)^2}_{\text{bias}^2} + \underbrace{\mathbb{V}_{\mathcal{D}}[h_{\mathcal{D}}]}_{\text{variance}} + \text{noise}.$$

In neural networks, model capacity increases with the number of layers and neurons. Low capacity → high bias (underfitting). High capacity → low bias but high variance (overfitting). Regularization and early stopping control the effective capacity, trading bias against variance.

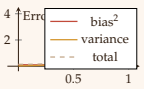

Figure 7: Bias–variance tradeoff: total error has a minimum at optimal capacity.

Universal Approximation Theorem

Theorem 0.1 (Cybenko, 1989) *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a continuous, bounded, non-constant activation function (e.g., sigmoid). Then for any continuous function $f : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exist $N \in \mathbb{N}$, vectors $\mathbf{w}_j \in \mathbb{R}^d$, scalars $v_j, b_j \in \mathbb{R}$ such that*

$$\sup_{\mathbf{x}\in[0,1]^d}\left|\sum_{j=1}^{N} v_j\,\sigma(\mathbf{w}_j\cdot\mathbf{x}+b_j) - f(\mathbf{x})\right| < \varepsilon.$$

The theorem guarantees that a single hidden layer (with enough neurons) can approximate any continuous function arbitrarily well. However, the required N may be astronomically large; depth allows more efficient representations.

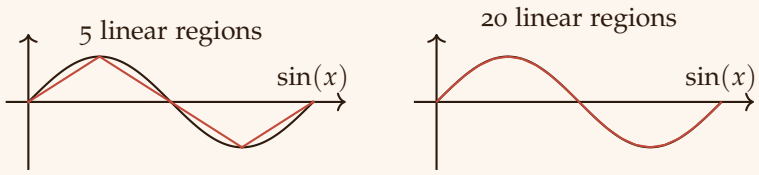

Figure 8: Approximating $\sin(x)$ with piecewise linear functions: more hidden units (linear regions) give better approximation, illustrating the universal approximation theorem.

Weight Initialization

Proper initialization is critical for deep networks. Common schemes:

- **Xavier (Glorot)**: $\mathrm{Var}[w] = \frac{2}{n_{\text{in}}+n_{\text{out}}}$ for tanh/sigmoid.
- **He (Kaiming)**: $\mathrm{Var}[w] = \frac{2}{n_{\text{in}}}$ for ReLU.

These keep signals in a reasonable range and prevent vanishing/-exploding gradients.